

🌸 Python Fundamentals 🌸 Part 4

Object Oriented Programming (OOP)

■ What is OOP?

OOP is a programming paradigm that organises code into **objects** — bundles of **data (attributes)** and **behaviour (methods)**. It makes code modular, reusable, and easier to maintain.

Term	Real-world analogy	In Python
Class	Recipe / Blueprint	Template for creating objects
Object / Instance	Actual cake baked from recipe	Concrete thing created from a class
Attribute	Ingredients list	Variables that store data
Method	Baking steps	Functions that define behaviour

■ *OOP is optional — but it shines when managing complex, structured data (like students, bank accounts, game characters).*

Class vs. Object

Class	Object
Blueprint / template	Concrete instance of the class
No memory until instantiated	Holds actual data, occupies memory
One class → unlimited objects	Each object is independent

```
class Car:
    brand = "Toyota"

car1 = Car() # create object
car2 = Car()
print(car1.brand) # Toyota (dot operator)
```

■ Constructor & self

A **constructor** is a special method that **auto-runs** when an object is created. It sets up the object with initial values using `__init__`.

self is a reference to the current object — Python passes it automatically, you never call it explicitly.

```
class Student:
    def __init__(self, name, gpa): # parameterised constructor
        self.name = name # instance attribute
        self.gpa = gpa

stu1 = Student("Rahul", 8.7)
stu2 = Student("Harshita", 9.1)
print(stu1.name, stu2.name) # Rahul Harshita
```

Constructor type	Has parameters?	Use when...
Default	Only self	All objects start with same values

🌸 Python Fundamentals 🌸 Part 4

Object Oriented Programming (OOP)

Parameterised	self + extra args	Each object needs unique values
---------------	-------------------	---------------------------------

■ *Python does NOT support constructor overloading — the last `__init__` defined wins!*

■ Attributes

Type	Where defined	Shared or unique?	Access via
Class attribute	Outside any method	Shared by ALL objects	ClassName.attr or obj.attr
Instance attribute	Inside <code>__init__</code> with self	Unique per object	obj.attr only

```
class Student:
    college = "ABC College" # class attribute – shared

    def __init__(self, name, gpa):
        self.name = name # instance attribute – unique
        self.gpa = gpa

s1 = Student("Ria", 9.2)
print(s1.college) # ABC College (via object)
print(Student.college) # ABC College (via class name)
print(s1.name) # Ria
```

■ Methods

Method type	Decorator	First arg	Can access
Instance method	(none)	self	Instance attrs + class attrs
Class method	@classmethod	cls	Class attrs only
Static method	@staticmethod	(none)	Neither — utility function

```
class Student:
    school = "ABC School" # class attr

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self): # instance method
        print(f"Name: {self.name}, Marks: {self.marks}")

    @classmethod
    def change_school(cls, new): # class method
        cls.school = new

    @staticmethod
    def add(a, b): # static method
        return a + b
```

■ The 4 OOP Pillars

Pillar	One-liner
Encapsulation	Bundle data + methods; control access
Abstraction	Hide complexity; show only what's needed
Inheritance	Child class reuses parent class code
Polymorphism	Same method name, different behaviour

■ Encapsulation

Bundling data and methods together, and **controlling who can access them**. Python uses naming conventions (not strict enforcement).

Access level	Prefix	Rule of thumb	Accessible outside?
Public	name	Use freely	✓ Yes
Protected	<code>_name</code>	Intended for internal / subclass use	■ Yes (but avoid)
Private	<code>__name</code>	Name-mangled to <code>_Class__name</code>	✗ Not directly

```
class Bank:
    def __init__(self, balance):
        self.owner = 'Alice' # public
        self._limit = 10000 # protected
        self.__bal = balance # private → _Bank__bal

b = Bank(5000)
print(b.owner) # Alice ✓
print(b._limit) # 10000 ■ allowed but not recommended
# print(b.__bal) # ERROR
print(b._Bank__bal) # 5000 name-mangled form
```

Getters & Setters — the clean way to access private data

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary # private

    def get_salary(self): # getter
        return self.__salary

    def set_salary(self, new): # setter
        self.__salary = new

e = Employee(50000)
print(e.get_salary()) # 50000
e.set_salary(60000)
```

■ *Python access modifiers are conventions, not enforced like Java/C++. Be disciplined!*

■ Inheritance

A **child class** inherits attributes + methods from a **parent class**, enabling code reuse and cleaner design.

Type	Structure	Example
Single	Parent → Child	Teacher inherits Employee
Multi-level	GP → Parent → Child	Accountant → AdminStaff → Employee
Multiple	Parent1 + Parent2 → Child	TA inherits Teacher AND Student

Single Inheritance

```
class Employee:
    start_time = "9AM"
    end_time = "5PM"

class Teacher(Employee): # Teacher IS an Employee
    def __init__(self, subject):
        self.subject = subject

t = Teacher("Data Science")
print(t.subject, t.start_time) # Data Science 9AM
```

Multi-level Inheritance

```
# Employee → AdminStaff → Accountant
class AdminStaff(Employee):
    def __init__(self, role):
        self.role = role

class Accountant(AdminStaff):
    def __init__(self, salary, role):
        super().__init__(role) # call parent __init__
        self.salary = salary

acc = Accountant(50_000, "CA")
print(acc.salary, acc.role, acc.start_time)
```

Multiple Inheritance

🌸 Python Fundamentals 🌸 Part 4

Object Oriented Programming (OOP)

```
class Teacher:
    def __init__(self, salary): self.salary = salary

class Student:
    def __init__(self, gpa): self.gpa = gpa

class TA(Teacher, Student): # inherits BOTH
    def __init__(self, name, salary, gpa):
        super().__init__(salary) # calls Teacher.__init__
        Student.__init__(self, gpa) # explicitly call Student
        self.name = name

ta = TA("Rahul", 50_000, 7.5)
print(ta.name, ta.salary, ta.gpa)
```

■ *super() calls the parent class method — super().__init__() lets you chain constructors cleanly.*

■■ Abstraction

Hide complex implementation details — show only what the user needs. Like driving a car: you press the brake; you don't need to know the hydraulics.

Concept	Detail
Abstract Class	Cannot be instantiated directly; acts as a blueprint for children
Abstract Method	Declared in parent with @abstractmethod; children MUST override it
Import needed	from abc import ABC, abstractmethod

```
from abc import ABC, abstractmethod

class Animal(ABC): # abstract class
    @abstractmethod
    def make_sound(self): # abstract method — no body
        pass

class Lion(Animal):
    def make_sound(self): # MUST override
        print("Roar!")

class Cow(Animal):
    def make_sound(self):
        print("Moo!")

lion = Lion(); lion.make_sound() # Roar!
cow = Cow(); cow.make_sound() # Moo!
```

■ *You CANNOT do Animal() — abstract classes can't be instantiated. Only concrete child classes can!*

■ Polymorphism

Same method name, **different behaviour** depending on the object/context. (poly = many, morph = forms)

🌸 Python Fundamentals 🌸 Part 4

Object Oriented Programming (OOP)

Type	Description	Example
Operator Overloading	Same operator, different operation based on types	1+2 gives '1'+2='12'
Method Overriding	Child provides its own version of a parent method	Dog.sound() vs Animal.sound()
Duck Typing	Python doesn't check type — just calls the method	Any object with .speak() works

Method Overriding

```
class Animal:
    def sound(self): print("Some generic sound")

class Dog(Animal):
    def sound(self): print("Bark") # overrides parent

a = Animal(); a.sound() # Some generic sound
d = Dog(); d.sound() # Bark (child takes precedence)
```

Duck Typing

"If it looks like a duck and quacks like a duck, it must be a duck." — Python calls whatever method exists, without caring about the type.

```
class Dog: def speak(self): print('Bark')
class Cat: def speak(self): print('Meow')
class Robot: def speak(self): print('Beep Boop')

def make_it_speak(entity): # doesn't check type!
    entity.speak()

for e in [Dog(), Cat(), Robot()]:
    make_it_speak(e)
```

■ OOP Quick Cheat Sheet

Concept	Keyword / Syntax	Remember
Define class	class MyClass:	PascalCase for class names
Constructor	def __init__(self, ...):	Runs automatically on object creation
self	self.name = value	Refers to the current object instance
Class attribute	attr = value (inside class, outside methods)	Shared by all objects
Instance attribute	self.attr = value (inside __init__)	Unique per object
Instance method	def method(self):	Access via obj.method()
Class method	@classmethod / def m(cls):	Access via Class.method()
Static method	@staticmethod / def m():	No self or cls — utility only
Public	self.name	Free access everywhere
Protected	self._name	Convention: internal use
Private	self.__name	Name-mangled: _Class__name

✿ Python Fundamentals ✿ Part 4

Object Oriented Programming (OOP)

Getter / Setter	<code>def get_x() / def set_x()</code>	Controlled access to private data
Inheritance	<code>class Child(Parent):</code>	Child gets all parent attrs & methods
super()	<code>super().__init__(...)</code>	Call parent constructor or method
Abstract class	<code>class X(ABC):</code>	Can't instantiate; children must override
Abstract method	<code>@abstractmethod / pass</code>	Forces child to implement it
Method overriding	Same name in child class	Child method takes precedence
Duck typing	<code>entity.speak()</code>	Python calls method without type check

■ OOP = Classes are the recipe ■, Objects are the dish ■■, Attributes are ingredients, Methods are the steps!